

Commandes — bootstrap serveur cliente

Serveur : `gathe@169.58.69.78` · Repo : `/home/gathe/gathe-finance` · À faire une seule fois, dans l'ordre. Màj 30/07/2026.

Légende : **SERVEUR** sur le VPS cliente **GIT/CI** côté GitHub (je peux le faire) **TOI** valeurs à fournir

Prérequis déjà OK : config GitHub (variables + secrets `CLIENT_*`) posée ✓ · DNS des 5 sous-domaines → `169.58.69.78` ✓ · images `:main` buildées ✓. **Manque** : le repo cloné + `.env.prod` sur le serveur (ci-dessous).

0. Authentification — où faut-il un PAT ?

Deux machines distinctes, deux logiques :

Machine	Auth	PAT nécessaire ?
Ta machine (tu as fait <code>gh auth login</code>)	compte <code>gh</code> rattaché	Non — les commandes <code>gh</code> (variables, <code>gh workflow run</code>) marchent sans PAT.
Serveur cliente (<code>169.58.69.78</code>)	aucune (machine séparée)	OUI — un PAT pour : ① <code>docker login ghcr.io</code> (tirer les images privées, obligatoire) ② <code>git clone</code> du repo privé.

Un seul PAT avec les scopes `repo` + `read:packages` couvre les 2 usages serveur. Ton `gh` local n'authentifie pas le serveur (machine différente).

1. Connexion + clone du repo **SERVEUR**

```
ssh gathe@169.58.69.78

# repo privé -> clone avec un PAT GitHub (scope repo + read:packages)
git clone https://<TON_PAT>@github.com/EdwinTchakounte/GatheFinance_project.git /home/gathe/gathe-finance
cd /home/gathe/gathe-finance
```

TOI : créer le PAT dans GitHub → Settings → Developer settings → Personal access tokens.

2. Relevé de la DMZ (lecture seule) **SERVEUR**

```
bash infra/scripts/inspect-dmz.sh
```

→ Noter : `EDGE_NETWORK`, `DATA_NETWORK`, `TRAFFIK_CERTRESOLVER`, alias Postgres, endpoint MinIO (servent au \$4).

3. Créer notre base + notre bucket (chez eux) **SERVEUR**

```
# Postgres (dans LEUR instance)
psql -U postgres -c "CREATE DATABASE gathe_prod;"
psql -U postgres -c "CREATE USER gathe WITH PASSWORD '<mdp_fort>';"
psql -U postgres -c "GRANT ALL PRIVILEGES ON DATABASE gathe_prod TO gathe;"

# MinIO (dans LEUR instance) — client mc
mc mb local/gathe-media
mc admin user svcacct add local <compte_admin> # -> access key + secret
```

TOI : choisir le mot de passe Postgres + garder les clés MinIO (pour le \$4).

4. Renseigner infra/.env.prod

SERVEUR

```
cp infra/.env.prod.client.example infra/.env.prod
nano infra/.env.prod
```

Valeurs à remplir :

```
# --- Domaines (connus) ---
SITE_DOMAIN=app.gathe-finance.com
PORTAL_DOMAIN=portail.gathe-finance.com
ADMIN_DOMAIN=admin.gathe-finance.com
API_DOMAIN=api.gathe-finance.com
CMS_DOMAIN=cms.gathe-finance.com
DJANGO_ALLOWED_HOSTS=app.gathe-finance.com,portail.gathe-finance.com,admin.gathe-finance.com,api.gathe-finance.com
COOKIE_DOMAIN=.gathe-finance.com
PUBLIC_BASE_URL=https://api.gathe-finance.com

# --- DMZ (relevé au §2) ---
EDGE_NETWORK=.....
DATA_NETWORK=.....
TRAFFIK_CERTRESOLVER=.....
TRAFFIK_ENTRYPOINT=websecure

# --- Base (créée au §3) ---
DATABASE_URL=postgres://gathe:<mdp>@<alias_postgres>:5432/gathe_prod

# --- MinIO (créé au §3) ---
AWS_STORAGE_BUCKET_NAME=gathe-media
AWS_ACCESS_KEY_ID=.....
AWS_SECRET_ACCESS_KEY=.....
AWS_S3_ENDPOINT_URL=http://<alias_minio>:9000
AWS_S3_REGION_NAME=us-east-1

# --- Secrets Django (générés) ---
DJANGO_SECRET_KEY=..... # python -c "import secrets;print(secrets.token_urlsafe(64))"
REVALIDATE_SECRET=.....
DJANGO_SUPERUSER_USERNAME=admin
DJANGO_SUPERUSER_EMAIL=admin@gathe-finance.com
DJANGO_SUPERUSER_PASSWORD=.....

# --- Email Brevo + paiement + tag ---
BREVO_API_KEY=.....
DEFAULT_FROM_EMAIL="Gathé Finance <noreply@gathe-finance.com>"
CONTACT_NOTIFICATION_EMAIL=contact@gathe-finance.com
TARA_API_KEY= # vide = mode mock
GATHE_IMAGE_TAG=main # la cliente tire les images :main
```

5. Accès registre + droits Docker

SERVEUR

```
docker login ghcr.io -u EdwinTchakounte # coller le PAT
sudo usermod -aG docker gathe # si `gathe` pas déjà dans le groupe docker
# (se déconnecter/reconnecter pour appliquer le groupe)
```

6. Déclencher le déploiement GIT/CI

```
# une fois §1-5 faits, je relance le déploiement cliente  
gh workflow run deploy.yml
```

→ SSH cliente → `pull + up` des 5 apps (backend, qcluster, site, portal, admin) → leur Traefik route et émet les certificats.

7. Post-déploiement + vérification SERVEUR / GIT/CI

```
# seeds emails/CMS + AppSettings (sur le serveur)  
bash infra/scripts/post-deploy-seed.sh  
  
# vérification (depuis n'importe où)  
curl -sL -o /dev/null -w "%{http_code}\n" https://app.gathe-finance.com/  
curl -sL -o /dev/null -w "%{http_code}\n" https://api.gathe-finance.com/healthz/
```

Attendu : 200 partout. Les certificats HTTPS sont émis automatiquement par leur Traefik dès que le DNS pointe (✓) et que nos conteneurs sont sur le réseau `edge` avec leurs labels.

Ordre à respecter : §1 → §5 sur le serveur AVANT le §6. Sinon le déploiement échoue sur `.env.prod` absent (sans dommage : rien ne se déploie).